

Four Hands-On Builds for Production-Grade RAG, MCP and Observability

Four independent builds on an Azure virtual machine with GitHub Copilot — each one measuring a stage of the retrieval pipeline separately, so that when an answer is wrong you can say which stage failed.

Companion sessions	Module 1 — Tokenisation and context engineering · Module 2 — Document ingestion · Module 3 — Retrieval architecture · Module 4 — MCP · Module 5 — Evaluation and observability · Module 6 — Improvement patterns
Companion demonstration	Demo 1 — Industrial Service Knowledge Copilot
Format	Teams of four. Task 2 uses the corpus Task 1 produces, but a pre-built index is provided if you need one. Tasks 3 and 4 stand alone.
Toolchain	Azure Linux VM · GitHub Copilot (agent mode) · Python 3.11 · Azure OpenAI chat and embedding deployments · FAISS · BM25 · a CPU cross-encoder
Every task ends with	A table of numbers on the same questions, and a statement of which pipeline stage each number diagnoses.

The rule that governs all four tasks. When a retrieval system gives a wrong answer, the overwhelming majority of the time **the right passage was never retrieved**. The model then did exactly what it should — answered from what it was given. Teams respond by improving the prompt or changing the model, which fixes the last stage of a pipeline that failed at the second. Today you will measure the stages separately so that this mistake becomes impossible to make.

Task	Build	Where	What it makes undeniable
1	The Ingestion Bench Real public technical manuals	Labs 1 + 2	Naive extraction destroys tables silently. Fixed-size chunking cuts a safety procedure in half and the answer is still fluent.
2	Retrieval Strategy Benchmark Five strategies, one table	Lab 3	Dense retrieval fails on part numbers. Sparse fails on paraphrase. Neither failure is visible in the answer.
3	Three MCP Servers Contracts, boundaries, logging	Lab 4	A vague tool description causes wrong calls, and the failure looks like a model problem. Identity has to reach the server.
4	Fix a Weak Pipeline, and Watch It Ablation plus observability	Labs 5 + 6	Retrieve twenty, rerank, pass four: cheaper, faster and more accurate from one change.

The four tasks together produce every one of the eight Day 3 assignment deliverables — see the closing page. Readouts are given at the start of the session following each lab.

Before you start — environment and ground rules

One VM per team

If your Day 2 environment is still running you need only the additional packages in section 2 and the corpus in section 3.

1 · The virtual machine

Size	Standard_D4s_v5 — 4 vCPU, 16 GB RAM. The cross-encoder reranker runs on CPU comfortably at the scale used today. No GPU required.
Image	Ubuntu Server 24.04 LTS · 64 GB Premium SSD
Networking	SSH (22), Streamlit (8501) and FastAPI (8000) inbound, source restricted to the training-room public IP.

2 · The toolchain

```
source ~/.venv/bin/activate
sudo apt install -y poppler-utils tesseract-ocr
pip install pypdf pdfplumber pymupdf "unstructured[pdf]" \
    faiss-cpu rank-bm25 sentence-transformers tiktoken \
    "mcp[cli]" fastapi "uvicorn[standard]" httpx pydantic \
    streamlit plotly pandas numpy scikit-learn jsonschema openai python-dotenv
mkdir -p ~/d3t1 ~/d3t2 ~/d3t3 ~/d3t4
```

Extraction	pypdf for naive · pdfplumber for layout and tables · pymupdf for reading order and blocks · unstructured for a higher-level pipeline. Four libraries because Task 1 compares four levels.
Retrieval	faiss-cpu for dense · rank-bm25 for sparse · Reciprocal Rank Fusion in about fifteen lines of your own code.
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2 via sentence-transformers . Downloads once, roughly 80 MB, runs on CPU. Reranking 30 candidates takes well under a second.
Token counting	tiktoken . Count tokens rather than estimating them — Task 1 depends on real numbers.

3 · Azure OpenAI, and the corpus

```
# ~/.env
AZURE_OPENAI_ENDPOINT=https://<your-resource>.openai.azure.com/
AZURE_OPENAI_API_KEY=<key>
AZURE_OPENAI_API_VERSION=2024-10-21
AZURE_OPENAI_CHAT=gpt-4o-mini
AZURE_OPENAI_EMBED=text-embedding-3-small
```

Today needs **two corpora**, for two different reasons.

Corpus	What it is	Why this one
A · Real technical PDFs	The US Department of Energy Fundamentals Handbooks — <i>Mechanical Science</i> volumes 1 and 2, covering pumps, valves, heat exchangers and diesel engines. Public domain, several hundred pages, genuinely industrial.	They contain exactly what breaks naive extraction: multi-column pages, torque and specification tables, numbered safety procedures, figure callouts, and repeated headers and footers. You cannot demonstrate an extraction failure on clean text.
B · The synthetic bulletin corpus	The 300-document service-bulletin corpus generated on Day 2, with its 40-question gold set naming the answering document.	Retrieval quality is only measurable when you know which passage is correct. Corpus A gives you realistic failures; corpus B gives you a scoreboard.

```
mkdir -p ~/corpus_a && cd ~/corpus_a
wget -O mechscience_v1.pdf "https://www.standards.doe.gov/standards-documents/1000/1018-bhdbk-1993-v1/@images/file"
wget -O mechscience_v2.pdf "https://www.standards.doe.gov/standards-documents/1000/1018-bhdbk-1993-v2/@images/file"
# If VM egress is blocked, the facilitator has both PDFs on the share.
# If you did not complete Day 2, regenerate corpus B with the prompt in Task 2, Step 1.
```

4 · Working with Copilot — the discipline that separates the top teams

Three rules for Copilot use in these labs.

1. **Never let it choose the chunk size for you.** Copilot defaults to fixed-size chunking with a round number, because that is what most tutorials do. Fixed size is the baseline you are measuring *against*, not the answer.
2. **Ask it to show you what was retrieved, not what was answered.** After it builds a pipeline, prompt: *"Print the exact passages passed to the model, with their scores and token counts, before the answer."* Almost every diagnosis today starts there.
3. **Make it write the tool description twice.** In Task 3, ask for a description written for the implementer, then one written for a competent new colleague who has never seen the system. Compare which one the model actually uses correctly.

5 · How your work is scored — identical for all four tasks

Criterion	Points	What full marks looks like
The system runs	15	The facilitator asks a question and gets an answer with its retrieved passages visible.
Stages measured separately	25	Retrieval quality and generation quality reported as different numbers, on the same questions.
Every comparison is like-for-like	20	Same corpus, same questions, one variable changed at a time.
Cost and latency reported alongside quality	15	Tokens and milliseconds per query for every configuration, not only the best one.
The failure is shown, not described	15	A specific wrong answer, traced to the specific extraction, chunk or retrieval defect that caused it.
Honest negatives	10	Changes that did not help are reported with their numbers. A table where every row is an improvement is not believed.

Automatic deduction: minus 10 for any quality claim without the retrieved passages available for inspection; minus 10 for any answer-quality number reported without its cost. These are the two habits Day 3 exists to break.

Task 1 — The Ingestion Bench

Labs 1 and 2

MODULES 1 & 2

EXTRACTION LEVELS

CHUNKING

TOKEN BUDGET

THE CUT PROCEDURE

REAL PDFS

The scenario

Everything downstream is capped by this stage. If the torque table was destroyed at extraction, no retrieval strategy will find it, no reranker will rescue it, and no prompt will make the answer right. This is the least glamorous work in the pipeline and it is where the largest single accuracy gain usually is.

You will ingest the same real technical manual four different ways, ask it the same ten questions, and find at least one question that every method except the best one answers **confidently and wrongly**.

Step 1 · Write the ten questions first

Before extracting anything, open the PDF as a human and write ten questions with the answers you can see on the page. Structure them deliberately:

Question type	How many	What it tests
Answer is in a table cell	3	Whether the row-to-value relationship survived extraction
Answer is a numbered procedure	2	Whether steps stayed together and in order
Answer spans two columns on one page	2	Whether reading order was preserved
Answer needs the section heading for context	2	Whether hierarchy was captured
Answer is in a figure caption	1	Whether figures were discarded silently

Record, for each, the page number and the exact text that answers it. This is your gold set for the rest of the task.

Step 2 · Four levels of extraction

Level	Method	Specifically check
1	Naive text — <code>pypdf</code> <code>extract_text()</code>	Is the torque table readable? Is column order correct? Are procedures intact?
2	Layout-aware — <code>pymupdf</code> blocks in reading order	Does reading order match the printed page? Are headers and footers excluded?
3	Table-preserving — <code>pdfplumber.extract_tables()</code> , tables rendered as markdown rows	Can the system answer "what is the value for X on variant Y" correctly?
4	Metadata-rich — level 3 plus section hierarchy, document version, page, content type	Can you filter to one section? Can you exclude a superseded revision?

Do this before you build anything else. Print the extracted text for **one page containing a table**, at all four levels, side by side. Put it on the screen. Nothing else in this pack makes the case as fast, and it is the single image your team will show management next month.

At level 1 the table comes back as a run-on sentence: the numbers survive, the relationship between them and their row labels does not. A question about a specific part returns a passage that *contains* the right number somewhere among five others, and the model picks one. It is fluent, it is confident, and it is wrong roughly four times in five.

Step 3 · The 4 × 10 grid

Run all ten questions against all four extraction levels with everything else held constant — same chunk strategy, same retrieval, same prompt, same model. Produce the grid: forty cells, each correct or incorrect, judged against the answer you recorded in Step 1.

Then pick your most instructive failure and write it up in full: the question, the page it should have come from, what the extraction produced, what the system answered, and why it was wrong. One paragraph, and it is worth more than the grid.

The finding to expect and verify. Moving from level 1 to level 3 typically produces a larger accuracy improvement than any change to the model, the prompt or the embedding. If your grid shows otherwise, say so — but check first that your table questions really do require the row relationship.

Step 4 · Chunking — three strategies, and a procedure cut in half

Now hold extraction at level 3 and vary the chunking. Compare **fixed size** (256, 512 and 1024 tokens with 10% overlap) against **section-aware** (break on headings) and **parent-child** (search small chunks, return the enclosing section).

The demonstration to construct deliberately. Find a numbered safety or lock-out procedure in the manual. Choose a fixed chunk size whose boundary falls inside it — you can compute exactly where the boundaries land, so this is arithmetic, not luck. Then ask a question about performing that procedure.

The retrieved chunk begins at step 3. The answer omits isolation and verification entirely. It is fluent, complete-sounding, and it describes an unsafe procedure. Show the retrieved chunk next to the printed page. This is the single strongest argument for structure-aware chunking, and it is why section-aware or parent-child almost always beats fixed size on maintenance documentation — typically as a one-day change.

Record for each strategy: **retrieval hit rate** (was the gold passage in the top 5), **answer correctness**, **chunks per document**, and **whether procedures survived intact** — check the last one by eye on five procedures, not by metric.

Step 5 · Passage count, and the number that surprises everyone

Hold everything else and vary how many passages you pass to the model: **top 3**, **top 8**, **top 20**. Record answer quality, tokens per request, cost and latency for each.

Passages passed	Tokens per request	Answer quality
Top 3	measure it	measure it
Top 8	measure it	measure it
Top 20	measure it	measure it

What you will find, and you need to see it on your own data to believe it. Top 20 costs roughly three times top 5 and frequently answers worse. Two effects combine: relevant information sitting in the middle of a long context is recalled far less reliably than information at either end, and every irrelevant passage dilutes the signal. The pattern to carry out of today is **retrieve broadly, rerank precisely, pass few** — cheaper, faster and more accurate from one change.

Step 6 · The token budget, including the tax nobody notices

Instrument every request with `tiktoken` and produce a real budget table for your own pipeline.

Component	Your measured tokens	How to control it
System prompt and instructions		Written once, reviewed, versioned. Rarely the problem.
Tool and MCP schemas		Charged on every request , whether the tool is used or not. Attach only what is relevant.
Retrieved passages		The main lever. Rerank, then cut to three to five.
Conversation history		Summarise older turns rather than carrying them verbatim.
User question		Negligible.
Reserved for the answer		Must be reserved explicitly, or generation truncates mid-sentence.

Then measure the schema tax directly: attach **no tools**, then **three**, then **twelve**, and record tokens per request before any work is done. Twelve tools attached to every call is a permanent charge on every request in the system's life.

Also worth measuring, and it takes two minutes. Count the tokens in one of your part numbers — something like HX-4471-B . Alphanumeric identifiers fragment into six to nine tokens that carry no meaning individually. This costs you a little money and a lot of retrieval quality, and it is the reason sparse keyword search remains essential for technical content. Task 2 measures that consequence.

Acceptance criteria

- ☐ Ten questions written from the printed page before any extraction, with page numbers and exact answer text.
- ☐ One table page shown extracted at all four levels, side by side.
- ☐ The 4 × 10 grid complete, plus one extraction failure written up with the wrong answer it caused.
- ☐ A procedure demonstrably cut by a fixed chunk boundary, with the retrieved chunk shown against the printed page.
- ☐ Passage-count comparison with quality, tokens, cost and latency for 3, 8 and 20.
- ☐ Token budget table filled with measured values, including the tool-schema tax at 0, 3 and 12 tools.

Defend your result

"Level 1 extraction answered ___ of 10; level 3 answered _____. Our worst failure was _____, and it happened because _____. Fixed 512-token chunking cut _____ in half. Top 20 cost _____ × top 3 and scored _____."

Task 2 — Retrieval Strategy Benchmark

Lab 3

MODULE 3

DENSE VS SPARSE

HYBRID & RRF

RERANKING

METADATA FILTERING

NO-ANSWER DETECTION

The scenario

One corpus. One set of questions with known correct passages. Five strategies. One comparison table — and the purpose of that table is not to express a preference. It is to be shown to your management with numbers on it.

Step 1 · The corpus and the question set

Use corpus B — the synthetic service-bulletin corpus with its gold set, because retrieval quality is only measurable when you know which passage is correct. If you need to regenerate it:

COPILOT — CORPUS AND GOLD SET

```
Generate 300 markdown documents for an industrial air-compressor service organisation:
maintenance procedures, troubleshooting guides, service bulletins, parts catalogue entries.
YAML front matter: doc_id, title, doc_type, product_family (GA-30 | GA-75 | ZR-160),
revision, issued_date, status (active|superseded), supersedes, access_level.
Parts entries must contain realistic alphanumeric part numbers like HX-4471-B.
Then gold_questions.jsonl with 60 questions, each with the doc_id that answers it, split:
  20 conceptual   - "why does discharge temperature rise when the cooler fouls?"
  20 identifier   - "what is the torque spec for the mounting bolts on HX-4471-B?"
  10 paraphrase  - worded deliberately unlike the source document
  10 out-of-corpus - genuinely not answerable from these documents
Mark the class of each question. Do not build any index yet.
```

Why the question classes matter more than the count. A mixed question set is what exposes the difference between strategies. On conceptual questions alone, dense retrieval looks excellent. On identifier questions alone, sparse looks excellent. Averaged together with no class breakdown, the two look similar and you learn nothing. **Report every metric per class as well as overall.**

Step 2 · Build the five strategies

	Strategy	What to expect
1	Dense only — embeddings + FAISS	Good on conceptual and paraphrase; fails on part numbers and error codes, and the failure is invisible — you get a plausible passage about a <i>different</i> part
2	Sparse only — BM25	Good on identifiers; fails when the question is phrased differently from the document
3	Hybrid — both, fused with RRF	Better than either alone on a mixed set, usually clearly so
4	Hybrid + cross-encoder reranking	The best precision, at additional latency. Measure both.
5	Hybrid + reranking + metadata filter	Typically the best overall — <i>if</i> your metadata is good enough to filter on

Reciprocal Rank Fusion — the whole of it

$$\text{score}(d) = \sum \text{over lists } 1 / (k + \text{rank}(d \text{ in that list}))$$
 $k = 60$ by convention

Dense and sparse scores are not on comparable scales and normalising them reliably is difficult, so RRF ignores the scores entirely and uses only the ranks. It needs no tuning, which is most of why it is the default.

For strategy 4: retrieve 30 to 50 candidates by hybrid search, rerank with the cross-encoder, cut to the top 3 to 5, then generate. For strategy 5, add a filter on `product_family` and `status = active` before search — not after.

Step 3 · The table that is the deliverable

Strategy	Context recall	Context precision	Answer correct	p95 latency	Tokens / query
Dense only					
Sparse only					
Hybrid					
Hybrid + rerank					
+ metadata filter					

Fill the same table again, broken down by question class. The second table is where the argument lives.

The two retrieval metrics, and how to compute them without a library

context recall = fraction of questions whose gold passage appears in the top k
context precision = fraction of retrieved passages that are relevant

You know the gold document id for every question, so context recall needs no model and no judgement. Compute it first — it is the number that diagnoses the stage where most failures actually happen.

Step 4 · Metadata filtering — the underused lever

Filtering restricts candidates **before** search: product family, version, access level, date. It turns a hard ranking problem into an easy one by removing what should never have been a candidate at all.

Measure it properly by constructing the case it is for: ask a GA-30 question when the corpus also contains a near-identical GA-75 procedure. Without the filter, the wrong variant is returned confidently. With it, the wrong variant was never a candidate.

Why it is underused. It requires metadata captured at ingestion, which is Task 1's level 4. Teams reach for a better embedding model instead, because that is a change they can make without revisiting the pipeline. Frequently the largest single improvement available, and almost always deferred.

Step 5 · No-answer detection, and the ten questions that expose it

You wrote ten out-of-corpus questions in Step 1. Run them against all five strategies now.

Most teams discover their system answers all ten confidently. Without no-answer detection, retrieval always returns its top matches however weak, the model answers anyway, and the user receives a fluent, confident, wrong answer — *with a citation to a document that does not support it*, which makes it worse rather than better.

Implement it as two things together: a threshold on the reranker score, and an explicit instruction that the model must answer only from the provided passages and must say so when they are insufficient. Then measure **no-answer accuracy** — how often it correctly declines — and report it beside the other metrics. It is almost never measured, and it is the difference between a system engineers trust and one they abandon.

Step 6 · One diagnosis you can run in front of the room

Take three questions your best strategy still gets wrong and diagnose each with two numbers only:

Context recall	Faithfulness	Diagnosis	What to fix
Low	—	Retrieval failed. The answer never had a chance.	Chunking, hybrid, reranking, metadata filtering
High	Low	The right passages were found and ignored.	Prompt grounding, citation enforcement, verification
High	High, still wrong	The source document itself is wrong or outdated.	Versioning and supersession — a content problem, not a system one
High	High, correct	The system is working.	Measure cost and latency; examine failing slices

This converts "the RAG system gives bad answers" into a specific diagnosis in one step. The overwhelming majority of cases land in the first row.

Acceptance criteria

- ☐ All five strategies run on the identical corpus and question set, one variable changed at a time.
- ☐ Both tables complete — overall, and broken down by question class.
- ☐ Latency and tokens per query reported for every strategy, not just the winner.
- ☐ The wrong-variant case constructed and shown with and without the metadata filter.
- ☐ No-answer accuracy measured on the ten out-of-corpus questions.
- ☐ Three residual failures diagnosed using the recall/faithfulness table.

Defend your result

"On identifier questions dense scored ____ and sparse ____; hybrid scored _____. Reranking added ____ ms and ____ points of precision while cutting tokens by _____. Our system declined correctly on ____ of 10 out-of-corpus questions before we added detection, and ____ of 10 afterwards."

The scenario

A knowledge assistant that retrieves technical documents, queries a service-ticket store and calls a diagnostic lookup. Three capabilities, owned by three different teams in real life. You will build all three as MCP servers, then demonstrate the two failures that make tool integration hard — neither of which is a model problem, though both look like one.

The arithmetic to lead with. Five AI applications across eight enterprise systems is forty bespoke integrations, each with its own authentication handling, error semantics, versioning and security review. Or eight governed servers, each built and reviewed once, consumed by any client. MCP's real value in an enterprise is that it turns integration from an application concern into a **platform** concern.

Step 1 · Build the three servers

Use the Python MCP SDK over stdio transport. Each server is a small, self-contained process — back them with SQLite or JSON files, because the point is the contract, not the storage.

Server	Exposes	Backed by
doc-search	<code>search_documents(query, product_family?, status?)</code>	Your Task 2 index — the retrieval component, reused
ticket-history	<code>find_tickets(asset_id, symptom?, since?)</code>	A generated table of 500 service tickets across 40 assets
parts-catalogue	<code>lookup_part(part_number)</code> · <code>find_parts_for(assembly)</code>	A generated catalogue with realistic part numbers

Step 2 · The tool contract — six elements, all of them

Element	Requirement	Common failure
Name	Unambiguous and specific	Two tools whose names could each plausibly apply
Description	What it does, when to use it, and when not to	Describes the implementation instead of the purpose
Input schema	Typed, each parameter described and constrained	Free-text parameters filled inconsistently
Output schema	Predictable structure	Raw system responses with inconsistent shape
Error semantics	Defined responses for not-found, unauthorised, unavailable	Exceptions, leaving the model with nothing usable
Examples	One or two representative invocations	None — usage inferred from the name alone

The standard to write to. Write tool descriptions for a competent new colleague who has never seen the system. That is precisely the reader you have — and unlike a colleague, it will not ask a clarifying question.

Step 3 · The experiment — measure how much the description matters

This is the part of the lab that produces a number, and it is a number almost nobody has.

- Write **30 realistic user requests**, each with the tool that should be called and the arguments it should be called with.
- Run them with **deliberately vague descriptions** — implementation-focused, no "when not to use", no examples. Record tool-selection accuracy and argument accuracy.

3. Rewrite the descriptions to the standard above. Change nothing else — same model, same prompt, same servers.
4. Re-run. Record the same two numbers.

What this demonstrates, and why it is worth a slide in your own organisation. Tool descriptions are written for a model, not for a human, and they are the least-discussed cause of agent failure. A vague description produces a wrong call that looks exactly like a model reasoning failure — so teams respond by changing the model, and the problem persists. Your before-and-after numbers turn that from an opinion into evidence.

Step 4 · The authorisation boundary, and the confused deputy

Give each server an independent authorisation check. Then build the failure deliberately, because it is the classic vulnerability of this architecture and it is very easily introduced.

Design	What happens
Wrong — the server holds one high-privilege credential to the underlying system, and trusts the client to have checked	A low-privilege caller asks a question; the server executes with its own high privilege and returns data the caller may not see. This is the confused deputy , and nothing in the logs looks unusual.
Right — the caller's identity is propagated to the server, and the server makes its own authorisation decision	The same request is refused at the server, regardless of what the client did or did not check.

Demonstrate both. Show the same request returning restricted data in one configuration and being refused in the other.

Then add the remaining four controls: **tool allow-lists per role, enforced outside the model**; **full call logging** with caller, arguments, result and timing; **idempotency** on any state-changing call; and an **approval gate** on anything irreversible.

Tool authorisation derives from who the caller is, never from what the model read. If retrieved content could ever cause a tool to be invoked with authority the caller does not have, the boundary is in the wrong place. You will attack this directly later in the week; today you build the boundary that makes the attack fail.

Step 5 · Failure semantics and egress review

For each of the three servers, define and then **test** what is returned on: not found · unauthorised · timeout · downstream unavailable. Four cases, three servers, twelve tests. A tool that raises an exception leaves the model with nothing usable and it will improvise.

Then complete the **egress review**: for each tool, what data leaves your boundary when it is called, and to where. One row per tool. This is a five-minute exercise that has stopped real deployments, and it is easiest to do while the contracts are fresh.

Step 6 · When not to use MCP

Add one honest row to your deliverable: a capability in your own system that **should not** be an MCP tool, and why.

The distinction that stops overreach. Does the model decide whether this happens? If not, it is not a tool. A retrieval step that always runs before generation is a pipeline stage — wrapping it in MCP adds latency and indirection for nothing. Use MCP when the model chooses at runtime, when several applications need the same capability, or when another team owns it.

Acceptance criteria

- ☐ Three servers running, callable, each with a complete six-element contract.
- ☐ Tool-selection and argument accuracy measured on 30 requests, before and after the description rewrite.
- ☐ Confused deputy demonstrated and then fixed, with the same request shown succeeding and failing.
- ☐ Twelve failure-semantics tests passing, with defined responses rather than exceptions.
- ☐ Egress review table complete, one row per tool.
- ☐ One capability named that should *not* be an MCP tool, with the reason.

Defend your result

"Vague descriptions gave ____% tool-selection accuracy; rewritten ones gave ____%. We changed no model and no prompt. Our confused-deputy demonstration returned ____ before identity propagation and ____ after. The tool we decided should not be a tool is ____."

Task 4 — Fix a Weak Pipeline, and Watch It

Labs 5 and 6

MODULES 5 & 6

EIGHT METRICS

ABLATION

TRACES AND SPANS

OBSERVABILITY

HONEST NEGATIVES

The scenario

You start from a deliberately weak pipeline and apply five improvements one at a time, measuring after each. The output is an ablation table exactly like Day 1's — which tells you not only that the system improved, but **which change earned which part of the improvement**, and which changes earned nothing.

Step 1 · The baseline, built badly on purpose

Fixed 512-token chunking with no overlap · dense retrieval only · no reranking · no metadata filtering · top 20 passages · a prompt that does not constrain the model to the passages. If you completed Tasks 1 and 2 you already know this configuration performs poorly; build it anyway, because every delta is measured from it.

Step 2 · Instrument before you improve

Implement the **eight metrics**. Four are computable without a model because you have a gold set; four need the model as a judge. Do not skip the split — knowing which is which is what makes the numbers trustworthy.

Metric	Question it answers	Diagnoses
Context recall	Did retrieval find the passages needed?	Retrieval — the usual culprit
Context precision	Of what was retrieved, how much was relevant?	Retrieval — cost and lost-in-the-middle
Faithfulness	Is the answer supported by the passages?	Generation — hallucination
Answer relevance	Does the answer address the question?	Generation — drift and evasion
Citation accuracy	Do the citations actually support the claims?	Generation — the trust signal users check
Hallucination rate	How often are unsupported claims made?	Generation — what management asks about
No-answer accuracy	Does it decline correctly when coverage is absent?	System — almost never measured
Groundedness	Is every claim traceable to a source?	System — composite quality signal

Step 3 · Traces and spans — eight per request

Record a trace per request with one span per stage: **query received** · **query rewrite** · **retrieval** · **reranking** · **context assembly** · **tool calls** · **generation** · **post-processing**. Each span carries start time, duration, and its own inputs and outputs. Write them to JSONL — you do not need a platform for this.

The span to insist on. Context assembly — record exactly what was passed to the model, with token counts per passage. When a user reports a wrong answer, that span usually answers the question immediately: the passage was not there, or it was there in position fourteen of twenty. Almost no proof of concept records it. It is cheap to add and it will resolve most of your support questions in one look.

Step 4 · The ablation — five changes, measured one at a time

Step	Change	Record
0	Baseline as built above	All eight metrics, latency, cost
1	Section-aware chunking	Delta on every metric
2	Add hybrid retrieval	Delta on every metric
3	Add reranking, cut passages from 20 to 5	Delta, including the cost reduction
4	Add metadata filtering	Delta on every metric
5	Add prompt grounding and citation enforcement	Delta, especially faithfulness and no-answer accuracy

Two rules for the ablation, and they are what make it worth doing.

One change at a time. If you change four things and performance improves, you have learned almost nothing — you do not know which one to keep and which is adding complexity for free.

Report the negatives. Some of these will not help on your data. Query rewriting frequently distorts intent; metadata filtering does nothing if your metadata is thin. A change measured at -0.005 is a change you should *remove*, and finding one makes the system simpler, cheaper and faster at no measurable cost. An ablation table where every row is an improvement is not believed by anyone who has run one.

Step 3 deserves particular attention. Reranking usually gives the best return per day of engineering work available to a retrieval system, and cutting from 20 passages to 5 **reduces** cost and latency at the same time. Three improvements from one change is rare; note it explicitly when you see it.

Step 5 · Citation enforcement — and the validation step that makes it real

Prompt grounding instructs the model to answer only from the provided passages and to say so when they are insufficient. It is cheap and effective, and it **shifts the rate rather than eliminating the behaviour** — treat it as one control among several.

Citation enforcement goes further: every claim carries a reference, and then you **validate programmatically** that the cited passage was actually retrieved and does contain the claim.

The validation step is what makes it real. Unvalidated citations are frequently plausible and wrong, which is worse than no citation — the user checks, sees a reference, and trusts it.

The adoption argument, worth repeating to management. An engineer who can check the source will use a system that is right 85% of the time. An engineer who cannot check will not use one that is right 95% of the time, because a wrong answer they cannot detect is worse than no answer. Citation accuracy is an adoption metric as much as a quality metric.

Step 6 · The observability dashboard — eight panels, each with an alert

Panel	Metrics	Alert on
Quality	Recall, precision, faithfulness, citation accuracy from scheduled runs	Any metric below threshold
No-answer behaviour	Decline rate and correctness of declines	Decline rate falling toward zero
Retrieval health	Score distribution, empty-result rate, filter hit rates	Rising empty results
Tool calls	Invocation rate, failure rate, latency per tool	Failure rate above threshold
Cost	Tokens and cost per request, per user, per tool	Sudden rise in tokens per request
Latency	p50, p95, p99 broken down by span	p95 breaching the objective
Feedback	Ratings, escalation rate, corrections	Rising negative feedback
Error clusters	Failures grouped by cause, ranked by frequency × cost	A new cluster appearing

Tokens per request is a quality signal, not just a cost signal. A sudden rise usually means retrieval quality dropped — scores fell, so the system pulled in more passages to compensate, or the fallback path started firing. It is one of the earliest warnings available, and it is measurable **without any labelled data**, which makes it operationally valuable in a way most quality metrics are not.

Step 7 · The regression suite

Assemble one suite and wire its triggers: any prompt, model, embedding, chunking or retrieval change, plus a scheduled weekly run regardless. It contains the gold set, every fixed failure, the out-of-corpus questions, the per-role variants and the known-hard cases. **It only grows.**

Seed it now with every failure you found in Tasks 1, 2 and 3 — the extraction failure, the cut procedure, the wrong-variant retrieval, the vague-tool-description miscall. Four permanent test cases, earned today.

Acceptance criteria

- ☐ All eight metrics implemented, with the four that need no model clearly separated from the four that do.
- ☐ Traces recorded with eight spans, including context assembly with per-passage token counts.
- ☐ Ablation table complete, one change at a time, deltas on every metric plus latency and cost.
- ☐ At least one change reported as no help or negative, with its number.
- ☐ Citations validated programmatically, not merely produced.
- ☐ Dashboard with eight panels, each carrying its alert condition.
- ☐ Regression suite seeded with the four failures found earlier today.

Defend your result

"Baseline context recall was ____; after all five changes it is _____. The largest single contribution came from ____ (+____). ____ contributed nothing measurable and we removed it. Cutting from 20 passages to 5 changed quality by ____ and cost by _____."

The Day 3 assignment, and where each piece comes from

Reference

	Assignment deliverable	Where you built it
1	RAG architecture, drawn as a flow	Tasks 1–4 combined — ingestion, chunking, index, retrieval, reranking, generation, post-processing
2	Tokenisation strategy	Task 1, Step 6 — the token budget per component, with the passage count justified by your own Step 5 measurement
3	Chunking and metadata strategy	Task 1, Steps 2 and 4 — the method and size you chose, and the metadata fields captured with what each enables
4	Retrieval benchmark	Task 2, Step 3 — the five-strategy table with recall, precision, latency and cost
5	MCP integration plan	Task 3 — three tool contracts, authorisation boundaries, failure behaviour, logging and egress review
6	RAG evaluation design	Task 4, Steps 2 and 7 — the eight metrics, gold set construction, and regression suite triggers
7	Observability dashboard design	Task 4, Step 6 — eight panels with metrics and alert conditions
8	Top five changes that improved accuracy	Task 4, Step 4 — from the ablation table, with the measured delta and the cost of each

Scored on architecture soundness, measurement rigour, integration design quality and observability completeness. Deliverable 8 carries into the Day 5 capstone.

Building your own evaluation set — six rules

- 1 **Real questions first.** From support tickets, engineer queries and search logs. Invented questions are systematically easier than real ones.
- 2 **Record the correct passage, not just the answer.** This is what makes context recall measurable, and what separates a diagnostic evaluation set from a score you cannot act on.
- 3 **Include out-of-corpus questions.** Ten to twenty the documentation genuinely does not cover.
- 4 **Include per-role variants.** If retrieval is access-filtered, the correct answer differs by role.
- 5 **Include known-hard cases.** Ambiguity, multi-document questions, superseded content.
- 6 **Synthetic questions to fill gaps** — with an expert verifying every correct answer.

The thirteen improvement levers, ranked — work top-down

Lever	Typical return	Effort	Note
Better chunking	High	Days	Section-aware over fixed. Often the largest single gain
Better metadata	High	Days–weeks	Enables filtering; often the largest gain after chunking
Hybrid retrieval	High	Days	Close to mandatory for technical content
Reranking	High	Days	Best return per day of work on this list
Citation enforcement	High	Days	Large trust gain; makes wrong answers checkable
Prompt grounding	Medium-high	Hours	Answer only from passages; say so when insufficient
Domain glossary	Medium-high	Days	Where field vocabulary differs from documentation
Failure-based tests	Medium-high	Ongoing	Prevents regression rather than improving the score
Query decomposition	Medium	Days	Only helps on genuinely compound questions
Answer verification	Medium	Days	Second pass checking claims. Doubles cost
Better embeddings	Medium	Weeks	Requires a full re-index. Measure before committing
Human feedback loop	Medium, compounding	Weeks	Slow to start, largest long-run source of gain
Knowledge graph	Variable	Months	Powerful for relationships, frequently premature

The ranking is deliberately ordered against what people instinctively try first. Most teams reach for better embeddings or a knowledge graph while chunking is still fixed-size and there is no reranker.

Data and references

Item	Reference
DOE Fundamentals Handbooks Corpus A, Task 1	<i>Mechanical Science</i> , DOE-HDBK-1018/1-93 and 1018/2-93, US Department of Energy, January 1993. Public domain. Covers diesel engines, heat exchangers, pumps and valves, with the multi-column layout, specification tables, numbered procedures and figure callouts that break naive extraction. standards.doe.gov
Cross-encoder reranker Task 2	cross-encoder/ms-marco-MiniLM-L-6-v2 . Runs on CPU; reranking 30 candidates is well under a second at this scale.
Reciprocal Rank Fusion	Combines ranked lists using the reciprocal of each item's rank rather than its score, because dense and sparse scores are not on comparable scales. $k = 60$ by convention; needs no tuning.

On data hygiene. Corpus A is public-domain US government documentation; corpus B is generated for the exercise. No Atlas Copco documentation should be placed on a training virtual machine. The scenarios are framed in Atlas Copco terms because the framing is what makes the failures recognisable.

The five sentences worth taking out of the room

- ❶ Most retrieval failures are retrieval failures — not model failures, not prompt failures.
 - ❷ Ingestion quality caps everything downstream. No later stage recovers what extraction destroyed.
 - ❸ Retrieve broadly, rerank precisely, pass few. Cheaper, faster and more accurate from one change.
 - ❹ Measure retrieval and generation separately, or you cannot tell which one failed.
 - ❺ Declining to answer is a feature, not a failure. A system that never says "I don't know" has not been measured on questions it cannot answer.
-

Before you leave today

Name, as a team, which of the thirteen levers you will apply first to your own system. Most teams name chunking or reranking, which is the right answer — and it is worth noticing how far that is from where the instinct started this morning.